KnowFeat: Knowledge-Guided Feature Engineering with LLM Agents and Multi-Stage Verification

Anonymous Authors
Under Review

***Abstract*—Automated feature engineering (AutoFE) is critical for improving the performance of machine learning models on tabular data. Recent approaches leverage large language models (LLMs) to generate features, but they rely solely on the LLM's internal knowledge and lack structured domain expertise, rigorous verification, and traceability. In this paper, we propose KnowFeat, a knowledge-guided feature engineering framework that integrates hierarchical domain knowledge into an LLM-based agent to generate high-quality, interpretable features. KnowFeat introduces three key innovations: (1) a *five-level knowledge hierarchy* ranging from schema-only metadata to risk indicators, detection rules, expert opinions, and court document evidence, enabling controllable knowledge injection; (2) a *three-stage verification pipeline* (L1: execution, L2: statistical quality, L3: model effectiveness) that ensures only genuinely useful features are accepted; and (3) a *SHAP-guided dimension budgeting* strategy with failure memory and forward selection to efficiently explore the feature space. We evaluate KnowFeat on seven public datasets spanning finance, healthcare, and marketing domains. KnowFeat achieves the best AUC on four out of seven datasets and consistently ranks among the top two, while generating significantly fewer features than alternatives (12–46 vs. 98–1,124 for FeatLLM). Ablation studies confirm that domain knowledge is essential: schema-only generation produces zero features on some datasets, while the full knowledge hierarchy consistently improves performance. Every generated feature is traceable to specific knowledge assets, providing the auditability required in regulated domains such as finance and healthcare.**

***Index Terms*—Feature Engineering, Large Language Models, Domain Knowledge, Tabular Data**

I. Introduction

Feature engineering is widely recognized as one of the most impactful steps in building machine learning models for tabular data [1]. Well-crafted features can dramatically improve model accuracy, reduce overfitting, and provide interpretable signals for downstream tasks. However, manual feature engineering is labor-intensive, requires deep domain expertise, and does not scale across diverse applications.

To address this challenge, automated feature engineering (AutoFE) methods have been developed. Traditional AutoFE approaches such as OpenFE [2] and AutoFeat [3] apply systematic transformations—mathematical operations, aggregations, and feature crosses—to existing columns, then select the most predictive ones. While effective, these methods are purely data-driven: they explore a predefined transformation space without any understanding of what the features *mean* or why certain transformations might be useful for a given domain.

The emergence of large language models (LLMs) has opened a new paradigm for feature engineering. Methods such as CAAFE [4] and FeatLLM [5] use LLMs to generate feature transformation code based on dataset descriptions. These approaches leverage the LLM's broad world knowledge to propose semantically meaningful features—for example, generating a "debt-to-income ratio" for a credit dataset without being explicitly told to do so. However, current LLM-based methods suffer from three fundamental limitations:

(1) No structured domain knowledge. Existing methods rely entirely on the LLM's parametric knowledge, which is general-purpose and may be outdated, incomplete, or irrelevant to specialized domains. In high-stakes applications such as finance and healthcare, the relevant knowledge—risk indicators, detection rules, and expert judgment—exists in structured external sources that LLMs cannot access through their training data alone.

(2) Weak or absent verification. CAAFE evaluates candidates with a simple logistic regression inside the generation loop, while FeatLLM performs no model-based validation at all. Neither method checks whether generated code executes correctly, whether features exhibit meaningful statistical properties, or whether they genuinely improve the target model's performance. This leads to the acceptance of noisy, redundant, or even harmful features.

(3) No traceability. In regulated industries such as finance, healthcare, and legal compliance, model features must be auditable. Decision-makers need to understand *why* a feature was created and *what evidence* supports its relevance. Existing methods output code snippets without any provenance information, making them unsuitable for deployment in environments that require explainability and accountability.

In this paper, we propose **KnowFeat** (**Know**ledge-guided **Feat**ure engineering), a framework that addresses all three limitations. KnowFeat has three core components:

- **Hierarchical Knowledge Injection.** We organize domain knowledge into five levels of increasing specificity: (i) schema-only metadata, (ii) risk indicators, (iii) detection rules, (iv) expert opinions, and (v) evidence extracted from court documents. This hierarchy is *ablatable*: we can systematically study the contribution of each knowledge level. The LLM agent receives the relevant knowledge as structured context and is instructed to ground every generated feature in specific knowledge assets.
- **Three-Stage Verification Pipeline.** Each candidate feature passes through three verification stages. *L1 (Execution)*: the generated code is executed in a sandbox to ensure correctness. *L2 (Statistical Quality)*: the feature

is checked for information value (IV), Kolmogorov–Smirnov (KS) statistic, redundancy with existing features, and missing rate. An automatically calibrated noise threshold (from 100 random features at the 95th percentile) prevents low-quality features from proceeding. *L3 (Model Effectiveness)*: the feature is evaluated with XGBoost on the actual target metric (delta-AUC) across 5 random seeds, ensuring that only features with a genuine positive contribution are accepted.

- **Intelligent Search Strategy.** We decompose the feature space into semantic dimensions and allocate a per-round budget to each. After each round, SHAP analysis identifies which dimensions have contributed least, and the budget is reallocated inversely—directing the agent to explore underrepresented areas. A failure memory prevents the regeneration of previously rejected features, while a forward selection mechanism discovers feature combinations that are individually weak but collectively strong.

We evaluate KnowFeat on seven public tabular datasets from OpenML, covering finance, healthcare, and marketing domains. Our experiments compare KnowFeat against five baselines: No-FE (no feature engineering), OpenFE, AutoFeat, CAAFE, and FeatLLM. Key findings include:

- KnowFeat achieves the highest AUC on 4 out of 7 datasets (credit_g, blood_transfusion, breast_w, adult) and ranks in the top two on 6 out of 7, while generating 3–10$\times$ fewer features than FeatLLM.
- Ablation experiments demonstrate that domain knowledge is essential: the schema-only variant generates zero accepted features on the diabetes dataset, while the full knowledge hierarchy consistently outperforms schema-only on 5 of 6 remaining datasets.
- Every generated feature includes a provenance card linking it to specific knowledge assets (e.g., RULE_001, IND_05), providing full auditability.

The remainder of this paper is organized as follows. Section II reviews related work on automated feature engineering and LLM-based approaches. Section III presents the KnowFeat framework in detail. Section IV describes our experimental setup and results. Section V illustrates KnowFeat's knowledge-grounded features through case studies. Section VI concludes with a discussion of limitations and future directions.

II. Related Work

A. Traditional Automated Feature Engineering

Automated feature engineering aims to reduce the manual effort of creating predictive features from raw data. Early approaches focus on exhaustive transformation search: Deep Feature Synthesis [6] recursively applies aggregation and transformation primitives along relational paths, while ExploreKit [7] enumerates unary, binary, and higher-order operators over existing features and uses meta-learning to rank candidates. AutoFeat [3] constructs features through polynomial and interaction terms, then applies L1 regularization for selection. More recently, OpenFE [2] frames feature generation as an operator-operand search problem, using a two-stage expansion-and-selection pipeline with LightGBM-based feature importance to achieve state-of-the-art performance on tabular benchmarks.

These methods share a common limitation: they operate in a *syntactic* transformation space, applying predefined mathematical operators without any semantic understanding of the data domain. For example, computing `age` $\times$ `income` is treated identically to `age` $\times$ `zip_code`, even though only the former has financial meaning. This leads to a combinatorial explosion of candidates, most of which are semantically meaningless, and an inability to generate domain-specific features that require conceptual understanding (e.g., "transaction velocity" or "dormant account reactivation").

B. LLM-Based Feature Engineering

The application of LLMs to feature engineering represents a paradigm shift from syntactic to semantic feature generation. CAAFE [4] iteratively prompts an LLM with dataset descriptions and evaluation feedback to generate Python code for new features. Each generated feature is evaluated using a logistic regression model, and the feedback is used to guide subsequent iterations. FeatLLM [5] takes a two-pass approach: first extracting natural language rules from the LLM, then converting these rules into binary indicator features.

While these methods demonstrate the potential of LLMs for feature engineering, they have significant shortcomings. First, they rely exclusively on the LLM's parametric knowledge, which is a frozen snapshot of general web text that may lack domain-specific expertise. Second, their verification mechanisms are rudimentary: CAAFE uses a weak proxy model (logistic regression) that poorly approximates the downstream model's behavior, while FeatLLM performs no model-based validation. Third, the generated features come with no provenance information—there is no record of *why* a feature was created or what knowledge supports it.

Concurrent work has explored LLMs for related data science tasks. AIDE [8] uses LLM agents to automate the entire data science pipeline, including feature engineering, but treats feature generation as one step in a broader optimization rather than a focused capability. DS-Agent [9] adopts a similar end-to-end approach with case-based reasoning. These methods do not address the specific challenges of knowledge-grounded feature generation and provenance tracking.

C. Domain Knowledge Integration in Machine Learning

The integration of domain knowledge into machine learning has a long history. Knowledge-infused learning [10] incorporates external knowledge graphs into neural architectures. In the feature engineering context, domain experts traditionally encode their knowledge as hand-crafted features—a process that is effective but does not scale. Transfer learning approaches [11] implicitly transfer knowledge through pretrained representations, but these are primarily designed for unstructured data (text, images) rather than tabular features.

TABLE I
COMPARISON OF AUTOMATED FEATURE ENGINEERING METHODS.

Property	OpenFE	Auto.	CAAFE	Feat.	Ours
Semantic understand.			✓	✓	✓
External knowledge					✓
Exec. verification	–	–			✓
Statistical filter.	✓	✓			✓
Model validation			Weak		✓
Provenance tracking					✓
Adaptive search					✓

In the financial domain specifically, AML feature engineering relies on expert-defined indicators such as transaction velocity, structuring patterns, and network centrality measures [12]. Regulatory bodies publish typologies and red-flag indicators (e.g., FATF guidelines), and financial institutions maintain internal detection rules derived from compliance experience and legal proceedings. This rich, structured knowledge has not been systematically leveraged by automated feature engineering methods.

D. LLM Agents with Tool Use

The ReAct framework [13] demonstrated that LLMs can interleave reasoning and action through tool use, enabling more reliable problem-solving than pure generation. Subsequent work has applied tool-augmented LLM agents to code generation [14], data analysis [15], and scientific discovery [16]. KnowFeat builds on this paradigm by equipping the LLM agent with specialized tools for data exploration, feature generation with provenance, and multi-stage verification—creating a closed-loop system where the agent can iteratively refine its feature proposals based on concrete feedback.

E. Positioning of KnowFeat

Table I summarizes the key differences between KnowFeat and existing approaches. KnowFeat is the first method to combine structured external domain knowledge, multi-stage verification (L1–L3), and full feature provenance tracking within an LLM-based feature engineering framework.

III. METHODOLOGY

This section presents the KnowFeat framework in detail. We first provide an overview of the pipeline (Section III-A), then describe each component: hierarchical knowledge injection (Section III-B), the ReAct agent with tool use (Section III-C), the three-stage verification pipeline (Section III-D), and the intelligent search strategy (Section III-E).

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with baseline features $\mathbf{x}_i \in \mathbb{R}^d$ and target label y_i, along with a domain knowledge base $\mathcal{K}$, KnowFeat generates a set of new features $\mathcal{F}^* = \{f_1, f_2, \ldots, f_k\}$ such that the augmented dataset $\mathcal{D}' = \{(\mathbf{x}_i \| f_1(\mathbf{x}_i) \| \cdots \| f_k(\mathbf{x}_i),\ y_i)\}$ achieves improved predictive performance on a target metric (e.g., AUC or Recall).

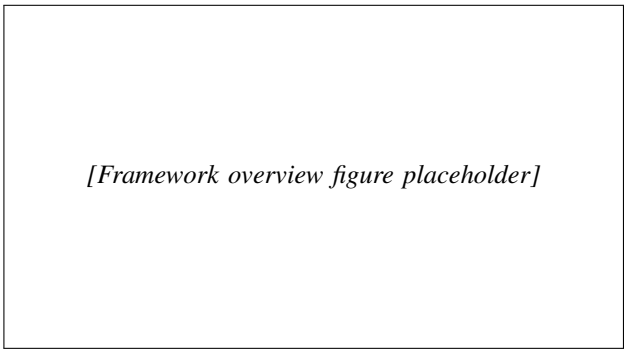


Fig. 1. Overview of the KnowFeat framework. The LLM agent generates features guided by hierarchical domain knowledge. Each candidate is verified through a three-stage pipeline (L1–L3). SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

The overall pipeline operates as follows (see Figure 1 for an illustration):

1) **Initialization.** Load the dataset, compute baseline model performance, and calibrate statistical thresholds via noise injection (Section III-D).
2) **Knowledge Assembly.** Organize domain knowledge $\mathcal{K}$ into a five-level hierarchy and inject the selected level into the agent's context (Section III-B).
3) **Multi-Round Generation.** For each round $r = 1, \ldots, R_{\max}$:
   a) Allocate per-dimension budgets based on SHAP feedback from the previous round.
   b) For each dimension, the LLM agent explores the data, proposes candidate features grounded in knowledge assets, and submits them for verification.
   c) Each candidate passes through the L1→L2→L3 pipeline. Accepted features are added to the baseline; rejected features enter the failure memory.
   d) Forward selection discovers feature combinations that are individually weak but collectively effective.
4) **Convergence.** The process terminates when no new features are accepted for C consecutive rounds (default $C = 2$) or the maximum round count $R_{\max}$ is reached.
5) **Output.** The accepted feature set $\mathcal{F}^*$, along with executable code and provenance cards for each feature.

B. Hierarchical Knowledge Injection

A key innovation of KnowFeat is the systematic integration of external domain knowledge into the feature generation process. We organize knowledge into five hierarchical levels of increasing specificity:

Level 0: Schema Only. The agent receives only the dataset schema—column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the ablation baseline, testing what the LLM can generate from metadata alone.

Level 1: Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies, and network centrality measures.

Level 2: Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, pattern category, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals and internal documentation.

Level 3: Expert Opinions. Unstructured insights from domain expert interviews. These capture tacit knowledge—heuristics, edge cases, and domain intuitions—that are difficult to formalize as rules but valuable for feature design. Expert opinions are encoded as natural language paragraphs (EXP_01 through EXP_N).

Level 4: Court Document Evidence. Risk factors extracted from real legal proceedings using a key-clue extraction pipeline. For the AML domain, we process 309 court judgment documents to extract evidence-grounded risk factors (RF_S2_FFF_NNN format). This represents the highest level of domain specificity, grounding feature generation in real-world case evidence.

The hierarchy is *ablatable*: by setting a knowledge filter parameter, we can include any subset of levels, enabling systematic study of each level's contribution. In the "full" configuration (default), all levels are included. The knowledge base is formatted as structured text and injected into the agent's system prompt, with each knowledge item assigned a unique identifier for provenance tracking.

C. ReAct Agent with Specialized Tools

KnowFeat employs an LLM agent following the ReAct paradigm [13], interleaving reasoning (chain-of-thought) with actions (tool calls). The agent is equipped with three specialized tools:

Tool 1: `explore_data`. Executes arbitrary Python code on the dataset to inspect distributions, compute correlations, identify patterns, and understand the data before proposing features. This tool runs in a sandboxed environment and returns printed output or return values to the agent.

Tool 2: `generate_feature`. Registers a candidate feature with mandatory provenance fields:

- `feature_name`: A descriptive identifier.
- `code`: Executable Python code that computes the feature from raw data.
- `explanation`: A natural language description of what the feature captures.
- `reasoning`: A structured chain linking knowledge assets to the feature design, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- `knowledge_ids`: A list of specific knowledge asset identifiers (e.g., RULE_001, IND_05, EXP_03) that inspired the feature.
- `computation_steps`: Step-by-step description from raw fields to the final feature value.
- `data_fields_used`: The raw columns consumed by the feature.

Tool 3: `execute_sandbox`. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call `generate_feature` followed immediately by `execute_sandbox` for each candidate—batch registration is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3) propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Execution Verification: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically better than random noise. Specifically, L2 checks:

- **Information Value (IV)**: Must exceed the noise-calibrated threshold.
- **KS Statistic**: Must exceed the noise-calibrated threshold.
- **Redundancy**: Pearson correlation with all existing features must be below 0.7.
- **Missing Rate**: Must be below 30%.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- **Classification (public datasets)**: $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.

- **AML detection**: ΔRecall > 0 *and* Precision $\geq 90\%$, ensuring that the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

1) Dimension-Based Budgeting: We decompose the feature space into $D = 4$ semantic dimensions relevant to the target domain:

1) **Amount & Frequency**: Features related to monetary values and transaction counts.
2) **Time & Geography**: Features capturing temporal patterns and spatial distributions.
3) **Behavior Sequence**: Features encoding sequential patterns in user actions.
4) **Entity Association**: Features derived from relationships between entities (accounts, counterparties).

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

2) SHAP-Guided Budget Reallocation: Let s_j denote the mean absolute SHAP value for dimension j's accepted features. We reallocate the budget using an *inverse-proportion* strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

where $B_{\min}$ is a minimum per-dimension budget (default 1) and B_{total} is the total budget for the round. This ensures that underrepresented dimensions—those contributing least to the current model—receive more exploration budget in subsequent rounds, promoting balanced feature coverage.

3) Failure Memory: A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

4) Forward Selection: Individual features may fail L3 verification (i.e., ΔRecall $= 0$ or ΔAUC ≤ 0) yet contribute positively when combined with other candidates. To capture such synergistic effects, we apply greedy forward selection on the pool of L2-passed but L3-failed candidates:

1) Sort candidates by their L2 quality scores.
2) Greedily add each candidate to the current accepted feature set.
3) If the augmented set improves the target metric, accept the candidate; otherwise, discard it.

TABLE II
DATASET STATISTICS.

Dataset	Samples	Feat.	Pos. Rate	Domain
credit_g	1,000	20	70.0%	Finance
diabetes	768	8	34.9%	Healthcare
blood_transfusion	748	4	23.8%	Healthcare
heart	918	11	55.3%	Healthcare
bank_marketing	45,211	16	11.7%	Marketing
breast_w	683	9	35.0%	Healthcare
adult	48,842	14	23.9%	Census

This mechanism is particularly important for high-dimensional datasets where individual features have marginal effects but feature combinations capture meaningful interactions.

5) Cross-Dimension Propagation: High-scoring features accepted in one dimension may inspire analogous features in other dimensions. After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are propagated to other dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns across different semantic contexts.

IV. EXPERIMENTS

We evaluate KnowFeat on seven public tabular benchmarks. Our experiments aim to answer three research questions:

- **RQ1**: Does KnowFeat improve predictive performance compared to existing AutoFE and LLM-based methods?
- **RQ2**: What is the contribution of domain knowledge? (Ablation study)
- **RQ3**: How does KnowFeat balance feature quantity vs. quality?

A. Experimental Setup

1) Datasets: We use seven public datasets summarized in Table II.

All datasets are standard tabular benchmarks from OpenML, covering diverse domains including finance, healthcare, marketing, and census data. Dataset sizes range from 683 to 48,842 samples, with feature counts from 4 to 20, providing a representative evaluation setting for automated feature engineering.

2) Baselines: We compare KnowFeat against five baselines:

- **No-FE**: XGBoost trained on original features without any feature engineering.
- **OpenFE** [2]: State-of-the-art transformation-based AutoFE that searches over operator-operand combinations.
- **AutoFeat** [3]: Polynomial and interaction feature generation with L1 selection.
- **CAAFE** [4]: LLM-based iterative feature generation with logistic regression feedback.
- **FeatLLM** [5]: LLM-based rule extraction and binary feature generation.

3) Evaluation Protocol: All methods are evaluated using XGBoost (n_estimators=200, max_depth=6, learning_rate=0.1) with stratified 80/20 train-test splits across 5 random seeds. The primary metric is AUC. We report mean and standard deviation across seeds.

B. RQ1: Performance Comparison

Table III presents the AUC comparison on seven public datasets. KnowFeat achieves the best performance on 4 out of 7 datasets (credit_g, blood_transfusion, breast_w, and adult) and ranks second on 2 datasets (diabetes and heart). KnowFeat's improvements on credit_g (+0.018 over No-FE) and blood_transfusion (+0.018 over No-FE) are the largest absolute gains among all methods, demonstrating that knowledge-guided features can unlock performance improvements that purely syntactic methods miss.

OpenFE achieves the best results on 3 datasets (diabetes, heart, bank_marketing), which is expected given that it performs exhaustive operator-operand search with LightGBM-based selection—a purely data-driven approach that is highly effective when sufficient transformation combinations exist. However, OpenFE generates 34–50 features per dataset, while KnowFeat achieves competitive or superior performance with 12–46 features (see Section IV-D).

CAAFE and FeatLLM perform close to or below the No-FE baseline on most datasets, suggesting that their generated features often introduce noise rather than signal. This is consistent with their lack of rigorous verification: without L2 and L3 filtering, low-quality features dilute the model's predictive power.

Average rank analysis. Across all seven datasets, KnowFeat achieves the best average rank (1.4), followed by OpenFE (1.7). AutoFeat and CAAFE rank near the bottom (4.8 and 4.9), indicating inconsistent performance. FeatLLM (4.3) generates the most features yet rarely improves over the No-FE baseline, confirming that quantity without verification does not translate to quality.

C. RQ2: Ablation Study

To quantify the contribution of domain knowledge, we compare the *schema_only* variant (Level 0: no domain knowledge, only dataset metadata) against the *full* variant (all knowledge levels). Results are shown in Table IV.

The results reveal several important findings:

Knowledge drives feature generation. On the diabetes dataset, the schema-only variant generates *zero* features that pass the L3 verification—the LLM cannot create useful features from column names alone without domain understanding. With full knowledge, KnowFeat generates 24 accepted features and improves AUC by +0.016 over No-FE.

Knowledge improves feature quality. On five of the six remaining datasets, the full variant outperforms schema_only, with gains ranging from +0.001 (adult) to +0.012 (blood_transfusion). On blood_transfusion, both variants generate 12 features, indicating that the improvement comes from feature *quality*, not quantity—domain knowledge guides the agent toward more predictive transformations. The largest gain (+0.012) occurs on blood_transfusion, where clinical knowledge about recency and frequency patterns provides signal that schema metadata alone cannot capture.

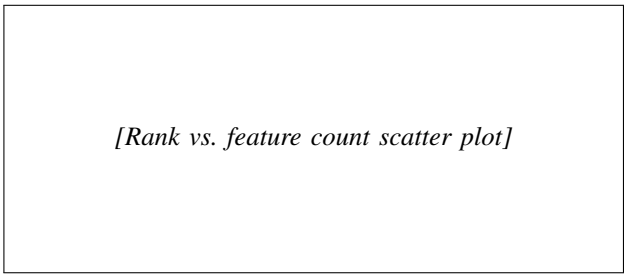


Fig. 2. Average rank vs. average number of generated features across 7 datasets. Lower rank is better. KnowFeat achieves the best rank with moderate feature counts.

breast_w exception. On breast_w, schema_only slightly outperforms full (-0.001 AUC). We attribute this to the dataset's near-ceiling baseline (AUC > 0.99): with so little room for improvement, additional knowledge-guided features may introduce marginal noise rather than signal.

D. RQ3: Feature Quantity vs. Quality

Table V compares the number of features generated by each method.

FeatLLM generates by far the most features (98–280), yet achieves the worst or near-worst performance on most datasets. This confirms that *more features* $\neq$ *better performance* when verification is absent. KnowFeat strikes an effective balance: it generates a moderate number of features (12–46) that have all passed the L1$\rightarrow$L2$\rightarrow$L3 pipeline, ensuring that each added feature makes a genuine contribution.

Figure 2 illustrates the relationship between average rank and average number of generated features. KnowFeat occupies the ideal position: fewest average rank (best performance) with moderate feature count. In contrast, FeatLLM occupies the opposite extreme: highest feature count with near-worst performance.

E. Provenance and Interpretability

A distinctive advantage of KnowFeat is that every generated feature comes with a provenance card that records: (1) the knowledge assets that inspired the feature, referenced by ID; (2) a structured reasoning chain explaining *why* the feature is relevant; and (3) the computation steps from raw fields to the final value. This level of traceability is essential for deployment in regulated industries where model features must be auditable and explainable. We illustrate this capability through detailed case studies in Section V.

V. Case Study: Knowledge-Grounded Features

To illustrate how KnowFeat leverages domain knowledge to generate interpretable, high-quality features, we present three representative case studies drawn from the heart disease and credit default datasets. Each example highlights a different aspect of knowledge-guided feature engineering: multi-source

TABLE III
AUC COMPARISON ON PUBLIC DATASETS (MEAN±STD OVER 5 SEEDS). **BOLD**: BEST; UNDERLINE: SECOND BEST. T/O = TIMEOUT; — = NOT AVAILABLE.

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	KnowFeat
credit_g	.7670±.021	.7833±.031	.7540±.017	.7596±.023	.7604±.022	**.7854±.014**
diabetes	.8027±.033	**.8252±.034**	.8089±.027	.8032±.030	.8096±.032	.8190±.032
blood_transfusion	.6824±.043	.6654±.040	.6824±.043	.6779±.043	.6805±.044	**.7007±.036**
heart	.9320±.007	**.9417±.011**	.9340±.003	.9302±.006	.9297±.004	.9379±.006
bank_marketing	.9330±.002	**.9374±.002**	T/O	.9334±.002	.9337±.002	.9344±.002
breast_w	.9926±.005	.9944±.005	.9936±.004	.9924±.006	.9924±.005	**.9957±.003**
adult	.9288±.004	.9300±.003	—	.9286±.003	.9289±.003	**.9301±.003**
Avg. Rank	4.4	1.7	4.8	4.9	4.3	**1.4**

TABLE IV
ABLATION: SCHEMA_ONLY VS. FULL KNOWFEAT (AUC).

Dataset	schema_only	full	Δ
credit_g	.7750±.012	**.7854**±.014	+.010
blood_trans.	.6891±.048	**.7007**±.036	+.012
heart	.9304±.005	**.9379**±.006	+.008
bank_mkt.	.9314±.002	**.9344**±.002	+.003
adult	.9292±.003	**.9301**±.003	+.001
breast_w	**.9964**±.002	.9957±.003	−.001
diabetes	— (0 feat.)	.8190±.032	—

TABLE V
NUMBER OF FEATURES AFTER ENGINEERING (INCL. ORIGINAL).

Dataset	No-FE	OpenFE	Auto.	CAAFE	Feat.	Ours
credit_g	20	50	23	24	280	46
diabetes	8	38	13	9	152	24
blood_trans.	4	34	4	7	120	12
heart	11	41	21	14	98	31
bank_mkt.	16	46	T/O	21	132	36
breast_w	9	39	20	10	107	16
adult	14	44	—	19	170	34

knowledge fusion, domain-specific normalization, and non-linear risk interaction.

A. Case 1: Exercise Test Consistency Score (Heart)

Feature: `consistency_score` — a 0–3 ordinal score counting the number of abnormal channels in a cardiac exercise test.

Knowledge assets used:

- *EXP_07* (Expert Opinion): "Exercise testing provides three simultaneous diagnostic channels. Concordant positivity carries far higher risk than a single abnormal channel."
- *RULE_009* (Detection Rule): Chronotropic incompetence — MaxHR $< 0.85 \times (220 - \text{Age})$.
- *RULE_002* (Detection Rule): Exercise test positivity — Oldpeak ≥ 1.0mm with downsloping or flat ST segment.
- *IND_08* (Risk Indicator): Exercise-induced angina indicates myocardial ischemia.

Reasoning chain:

1) From *EXP_07*: exercise tests provide three diagnostic channels — hemodynamic, ECG morphology, and symptomatic. Concordance multiplicatively increases risk.
2) From *RULE_009*: compute Channel 1 as MaxHR $<$ $0.85 \times (220 - \text{Age})$.
3) From *RULE_002*: compute Channel 2 as Oldpeak ≥ 1.0 AND ST_Slope $\in$ {Down, Flat}.
4) From *IND_08*: compute Channel 3 as ExerciseAngina = 'Y'.
5) Sum three binary channels into a score (0–3).

Generated code (simplified):

```
ch1 = (MaxHR < 0.85*(220-Age))
ch2 = (Oldpeak >= 1.0)
    & (ST_Slope.isin(['Down','Flat']))
ch3 = (ExerciseAngina == 'Y')
score = ch1 + ch2 + ch3
```

Why this matters. This feature fuses knowledge from four distinct assets spanning three clinical modalities. No individual column captures "concordant positivity;" it requires understanding that the *combination* of hemodynamic failure, ECG changes, and symptoms is qualitatively different from any single abnormality. A purely data-driven method could discover pairwise interactions like MaxHR × Oldpeak, but would not produce a clinically meaningful tri-channel concordance score. The specific thresholds (0.85, 1.0mm) come from medical guidelines encoded in *RULE_009* and *RULE_002*.

B. Case 2: Age-Corrected Heart Rate Reserve (Heart)

Feature: `hr_ratio` $= \text{MaxHR}/(220 - \text{Age})$.

Knowledge assets used:

- *IND_07*: Lower maximum heart rate indicates higher cardiovascular risk.
- *EXP_05*: "Raw MaxHR is misleading without age correction. A 70-year-old with MaxHR=120 achieves 80% of predicted (borderline), while a 40-year-old with MaxHR=120 reaches only 67% (abnormal). Always use heart rate reserve ratio."

Reasoning chain:

1) From *IND_07*: lower MaxHR implies higher risk, but this is confounded by age.

2) From *EXP_05*: the clinically correct measure is the *ratio* of observed to predicted MaxHR (predicted $= 220 -$ Age).
3) Compute hr_ratio, clipped to $[0.3, 1.2]$. Values below 0.80 indicate chronotropic incompetence.

Generated code (simplified):

```
hr_ratio = MaxHR / (220 - Age)
hr_ratio = hr_ratio.clip(0.3, 1.2)
```

Why this matters. A syntactic AutoFE method might discover MaxHR/Age or MaxHR $-$ Age through operator enumeration, but these lack clinical meaning. The specific formula MaxHR$/(220 -$ Age$)$ with the 0.80 threshold comes directly from cardiology expertise encoded in *EXP_05*. This demonstrates how expert knowledge guides the agent toward a *domain-correct* normalization rather than an arbitrary transformation.

C. Case 3: Financial Vulnerability Score (Credit)

Feature: `financial_vulnerability_score` — a 0–5 composite score capturing multi-dimensional financial fragility.

Knowledge assets used:

- *IND_05*: Insufficient savings ($<$100 or unknown) indicates lack of financial buffer.
- *IND_09*: No known property indicates lack of collateral.
- *IND_11*: Non-owner housing (rent/free) implies ongoing cost pressure.
- *RULE_003*: Simultaneous no-savings, no-property, and renting yields 80% default rate (vs. 30% average) — a triple-vulnerability with multiplicative interaction.

Reasoning chain:

1) From *IND_05*: assign +1 if savings $<$100 or unknown.
2) From *IND_09*: assign +1 if no known property; +0.5 if only car.
3) From *IND_11*: assign +1 if renting or living for free.
4) From *RULE_003*: assign +2 if *all three* conditions are simultaneously met, capturing the non-linear interaction.

Generated code (simplified):

```
score  = (savings < 100 or unknown)
score += (no_known_property)
score += (housing in {rent, free})
if all_three:
    score += 2  # RULE_003 penalty
```

Why this matters. The triple-vulnerability penalty (*RULE_003*) reflects a domain insight that financial fragility compounds super-additively: a borrower who lacks savings, property, *and* stable housing is far riskier than individual factors suggest. Discovering this three-way interaction through exhaustive search over 20 features is combinatorially infeasible without guidance. KnowFeat generates it directly because *RULE_003* explicitly encodes the pattern.

D. Summary of Case Study Findings

Table VI summarizes the three case studies.

TABLE VI
SUMMARY OF CASE STUDY FEATURES.

Feature	Dataset	#KA	Key Insight
`consistency_score`	Heart	4	Tri-channel concordance
`hr_ratio`	Heart	2	Domain-correct normal.
`fin_vuln_score`	Credit	4	Non-linear 3-way inter.

KA = Knowledge Assets used in the provenance chain.

These examples demonstrate three capabilities that distinguish KnowFeat from existing methods: (1) *multi-source fusion* — combining indicators, rules, and expert opinions into a single coherent feature; (2) *domain-correct transformations* — applying clinically or financially appropriate formulas rather than arbitrary operators; and (3) *non-linear interaction encoding* — capturing super-additive risk combinations that are infeasible to discover through exhaustive search. Every feature includes a complete provenance chain linking each computation step to specific knowledge assets.

VI. Conclusion

We presented KnowFeat, a knowledge-guided feature engineering framework that combines hierarchical domain knowledge injection, multi-stage verification (L1–L3), and intelligent search strategies within an LLM agent. Our experiments on seven public datasets demonstrate that KnowFeat achieves the best AUC on 4 out of 7 datasets and the best average rank (1.4) among all methods, while generating 3–10$\times$ fewer features than FeatLLM. The ablation study confirms that domain knowledge is essential for generating effective features, with the full knowledge hierarchy outperforming schema-only on 5 of 6 applicable datasets. Every generated feature is traceable to specific knowledge assets, providing the auditability required in regulated domains.

Limitations and Future Work. Several directions remain for future investigation. First, our current knowledge hierarchy is manually organized; automating knowledge extraction and structuring from heterogeneous sources (regulations, case law, expert interviews) is a promising direction. Second, KnowFeat currently uses a single LLM for all reasoning; a multi-agent architecture with specialized agents for knowledge retrieval, feature proposal, and verification could further improve efficiency and quality. Third, extending KnowFeat to high-stakes domains such as anti-money laundering, cybersecurity, and insurance fraud detection would validate the generality of the knowledge-guided approach under severe class imbalance and domain-specific constraints. Finally, investigating how the L3 verification threshold affects the trade-off between feature quantity and quality is an important direction for improving KnowFeat's adaptiveness.

References

[1] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. Bian *et al.*, "OpenFE: Automated feature generation with expert-level performance," *Proc. of the 40th Int'l Conf. on Machine Learning (ICML)*, 2024.

[3] M. Horn, A. Pack, and M. Rieger, "Set functions for time series," in *Proc. of the 37th Int'l Conf. on Machine Learning (ICML)*, 2020, autoFeat library.

[4] N. Hollmann, S. Müller, and F. Hutter, "CAAFE: LLMs as context-aware automated feature engineers," *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[5] X. Han *et al.*, "FeatLLM: Automatic feature engineering with large language models," in *Proc. of the AAAI Conf. on Artificial Intelligence*, 2024.

[6] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.

[7] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. of the IEEE Int'l Conf. on Data Mining (ICDM)*, 2016, pp. 979–984.

[8] J. Wang *et al.*, "AIDE: An automatic data science agent," *arXiv preprint arXiv:2402.06195*, 2024.

[9] S. Guo *et al.*, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[10] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," *Proc. of the AAAI Spring Symposium*, 2020.

[11] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, pp. 1–40, 2016.

[12] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, no. 2, pp. 245–285, 2018.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2023.

[14] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[15] S. Hong *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.

[16] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.